NANYANG TECHNOLOGICAL UNIVERSITY SINGAPORE
School of Civil and Environmental Engineering
College of Engineering

Introduction to Computational Thinking

Mini Project Assignment

Course: CV1014/AY25S2
Issue date: March 6, 2026
Presentation due dates: April 13-17, 2026

---

1 OBJECTIVES

The purpose of this assignment is to help you apply the computational thinking (CT) concepts you learned in this course by developing a mini-project. This project will strengthen your problem-solving and programming skills. Your program must demonstrate the four CT aspects covered in the course: **abstraction, decomposition, pattern recognition, and algorithm design**. You will also gain hands-on experience with Python, including **data processing, conditional statements (branching), loops, data structures, functions, and basic algorithm design**. Your program may include user interface features. This is not mandatory, but it is encouraged and may earn bonus marks.

2 THEMES FOR CHOICES

To reduce the risk of plagiarism, we provide a list of themes for you to choose from. Each group of **three students** must select **one topic** from the list below. **Note:** The topic "F&B Recommendation System" is only used as an example to illustrate the implementation steps. **Do not choose this topic for your mini-project.**

a) Campus lost-and-found management system.

b) Event ticket booking and management system for campus activities.

c) Gym and fitness centre recommendation system.

d) Textbook exchange platform that matches buyers and sellers within a university.

e) Flight recommendation system between multiple cities.

f) CEE EST Lab PC reservation and scheduling system.

g) Campus shuttle bus route and arrival-time recommendation system.

h) Campus club and activity matching system.

i) Construction worker shift scheduling and work allocation management system.

j) Vessel berth allocation and port docking schedule management system.

k) Smart lab inventory system to track chemicals, reagents, and equipment usage in NTU labs, with safety alerts and restocking suggestions.

l) NTU dormitory resource management system to track utilities and maintenance requests, and to suggest energy-saving practices.

m) Maritime safety training and incident management system to train NTU maritime students in emergency response and simulate real-world scenarios (e.g., oil spills, collisions).

n) Campus infrastructure sustainability tracker to monitor energy use, water consumption, and (optionally) structural condition indicators for NTU buildings.

o) Campus event finder, recommendation, and registration management system for university students.

p) Study group matching system based on course, schedule, and learning objectives.

q) Personal expense tracking and spending pattern analysis system with simple data visualisation.

r) Personalized travel itinerary generator based on traveller preferences, budget, and time constraints.

s) Personalized clothing style recommendation system based on user preferences.

The following functions can be implemented when developing each system:

a) User interface (UI) design (Optional: up to 10 bonus marks).

b) Database design to store the required information.

c) Adding new records and updating existing records in the database.

d) Searching for information in the database.

e) Sorting results using different criteria (e.g., distance, price, or rating).

f) Optional advanced features beyond the course scope are encouraged. Bonus marks may be awarded (up to 5 marks).

3 EXAMPLE MINI-PROJECT — THEME, DESIGN, IMPLEMENTATION

The following section is provided **only as an example** to help you understand how a mini-project can be defined and developed after a theme has been selected. It presents an example theme, the general idea of the program, and an example of how computational thinking, program design, and implementation may be approached.

You should use this example as a source of guidance and inspiration, **not as a fixed template**. After selecting your own theme, your group is expected to define the data, functions, and methods that are most appropriate for your proposed system. For example, some types of data and functions may be suitable for recommendation systems, while different data structures, program functions, and logic may be more appropriate for management systems, matching systems, inventory systems, and other project types.

Part of the mini-project evaluation is your ability to make appropriate design decisions based on the selected theme. This includes defining suitable information to store, selecting appropriate functions and methods, and showing creativity in the overall program design.

3.1 Example theme: F&B recommendation system

In this example, you will develop a Python program that recommends an NTU canteen based on the user's location and preferences. The program should include information for **at least 10 NTU canteens**. For each canteen, you may store relevant details such as **location, stall/booth names, signature dishes, price range, rating/rank, facilities/environment, and operating hours**. You may also include additional information if you wish.

This information should be stored in your system, and you may optionally allow users to update it. You can represent each canteen's location using **2D coordinates (row, column)**. A sample map image is shown below (Figure 1); you may use any suitable image in your project.

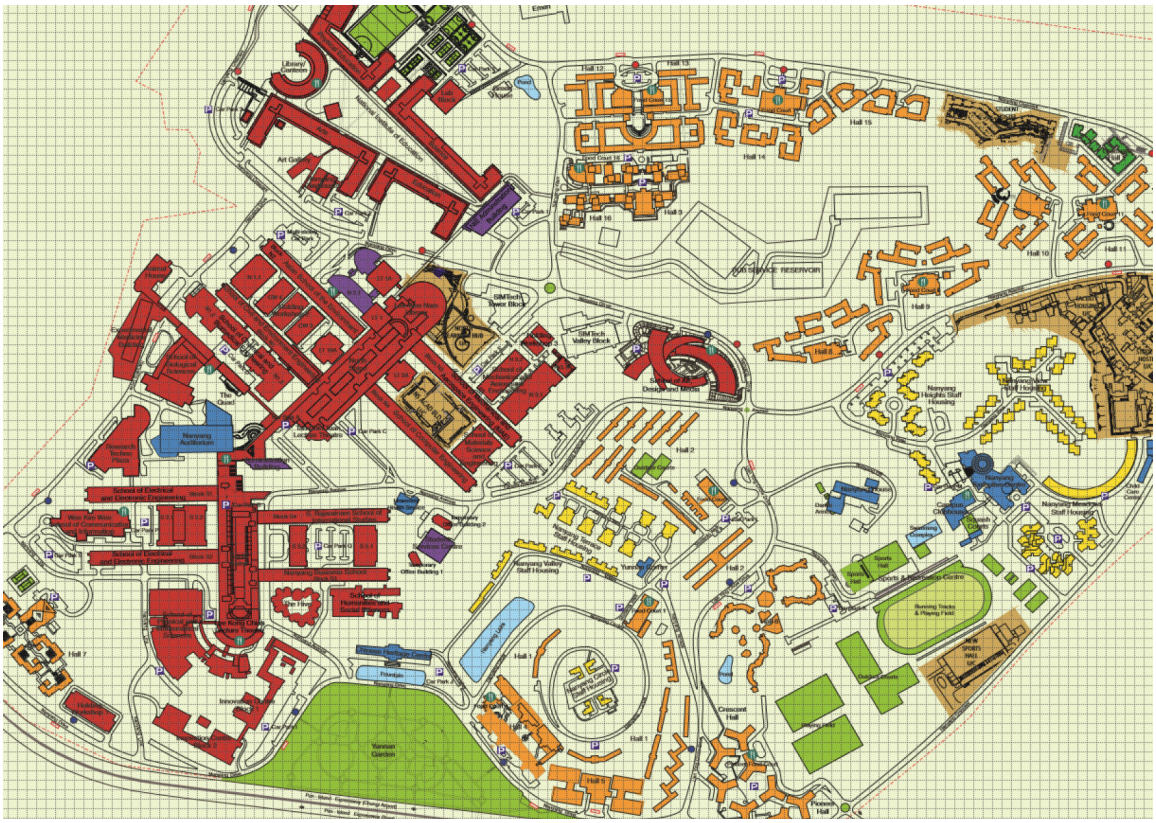

Figure 1: Sample map image.

3.2 Example computational thinking approach

A basic F&B recommendation system can work as follows:

i. Ask the user to enter their location on the NTU campus. If distance is relevant, represent the location using 2D coordinates (row, column).

ii. Ask the user to enter their search criteria (e.g., preferred food type, budget, or rating).

iii. Display the recommended canteen(s) and the relevant information.

You should also consider the following in your code:

1. **Decomposition:** Break the problem into major functions. For example:

    a. Set up the information system and pre-store the initial data.

   b. Display the map to the user (optional, if you use a map interface).
   c. Allow the user to update the stored information (optional).
   d. Collect the user's query and generate a recommendation.
   e. Sort results using criteria such as distance, price, or rank.

2. **Programming style:** Use clear variable and function names, and include appropriate comments.

3. **Testing:** Test your program with different inputs to verify that the results are correct.

3.3 Example program design

Before writing any code, you should carefully read the requirements and design your program using *pseudocode* (or a flowchart). At this stage, identify the main functions you will implement, what each function should do, what inputs it needs, what outputs it produces, and how the functions work together as one complete program.

Next, you can build a basic project skeleton. This initial version should focus on the user interface (i.e., how the user enters a request and receives a recommendation), the map display (if you use one), and the initialization of key variables and data.

3.3.1 Define and initialize data structures

You should define suitable data structures to store the required information, such as each canteen's location, operating hours, available food items, food prices, and canteen rating/rank.

3.3.2 Project skeleton

Table 1 lists example functions you may implement in your program. You are not limited to these functions; you may add or modify functions based on your design.

Table 1: Example functions for the mini-project (you may modify or extend this list).

Function	Description
`get_user_location()`	Get the user's location through console input or a mouse click.
`distance_a_b(location_of_a, location_of_b)`	Compute the distance between two locations.
`sort_distance(user_location, canteens_location)`	Sort canteens by distance from the user (ascending) and display the results.
`search_by_food(foodname, foodlist_canteens)`	Search canteens and return those that offer the requested food item.
`sort_by_rank(ranklist_canteens)`	Sort and display canteens by rank/rating.
`search_by_price(price, foodlist_canteens)`	Return food items (or canteens) within a given price range.
`mouse_click()*`	Optional: Return the coordinates of a mouse click on the map.
`update_information()*`	Optional: Allow users to update canteen information in the database.
`transport(user_location, dest_location)*`	Optional: Provide transport information from the user's location to a destination.

3.3.3 Map display

Displaying a map is not required for this mini-project. However, it can significantly improve the user interface and make your program easier to use. If you choose to include a map, you may use `Pygame` to display it. You may use any suitable map (e.g., an NTU map), either downloaded from the Internet or created by your group.

Reference: https://en.wikipedia.org/wiki/Pygame

3.4 Example program implementation

3.4.1 Data structures

Use appropriate and efficient data structures in your program, such as strings, lists, tuples, and dictionaries.

3.4.2 Coding

In your implementation, apply the programming techniques covered in this course, including data processing, conditional statements (branching), loops, data structures, functions, and basic algorithm design. Your program should also reflect the four CT aspects: abstraction, decomposition, pattern recognition, and algorithm design.

Each time you add a new feature, test your program thoroughly before moving on to the next one.

4 ASSIGNMENT GROUPING

To complete this mini-project, you must form a group of **three** students according to the instructions from your lab supervisor. Each group must select one of the proposed themes. Within each tutorial group, the same theme may be selected by at most **two** teams.

Groups must be formed within your own tutorial/lab slot. Use the shared spreadsheet to register your team: Shared team registration spreadsheet.

If a group has fewer than three members, it may be merged with another group. If you have not formed a team by the end of **Week 9 (22 March)**, the teaching team will assign groups randomly.

Once your group has finalized the program flow (e.g., using pseudocode or a flowchart), divide the work among team members so that the workload is balanced. Each member should implement specific parts of the program, and the final code should be integrated using modules (e.g., by importing Python files).

You must clearly state which parts were completed by each team member. This information will be used for both **individual assessment** and **team collaboration assessment**.

Note: Make sure your program runs on **Python 3.8 or later** (lab version).

5 SUBMISSION PROCEDURE AND ASSESSMENT

i. **Deadline.** The submission deadline for each group is **11:59 pm on the day of your last lab session in Week 13**. *Example:* If MS1 has a lab session on Monday (08:30–10:20), and the Week 13 lab date is 13 April, then the submission deadline for MS1 is **11:59 pm on 13 April**.

ii. **Items to submit.** Each group must submit the following three items:

   a) A zipped folder containing your Python code, labelled with your team ID (e.g., `CV1_1_code`, `CV1_2_code`). **Important:** A coding framework (`Framework.ipynb`) is provided. Please use it as a reference for your project.

   b) The **program assessment rubric** (see Appendix 1). Complete the *Description / Comment* fields as instructed in the appendix. **Important:** Submit this rubric **one day before** your presentation so that the tutor can use it during marking. Label it with team ID (e.g., `CV1_1_rubric`, `CV1_2_rubric`)

   c) Slides for your **6-minute presentation**. Label it with team ID (e.g., `CV1_1_pres`, `CV1_2_pres`)

iii. **Where to submit.** Submit all three items via **NTULearn** under `Content / Submissions / Mini project submission`. Only **one** member from each group should submit the files. Please do not submit multiple times from different group members.

iv. **Presentation and Q&A (Week 13).** Each group will deliver a **6-minute presentation**, followed by a **4-minute Q&A**. Your presentation should explain your computational thinking process, your program design, and include a short demonstration of your program using test inputs. The contribution of each team member must be clearly indicated. During the Q&A:

   a) Other students may ask questions. **Students who ask a question will receive up to 2 bonus marks.**

b) The instructor will use the assessment form to grade the overall performance. Scores will be released by the end of the week on NTULearn.

Important: Ideally, the presentation should be delivered by all team members. However, it is acceptable for one or two members to present if all members agree. In such cases, students who do not present are expected to take a greater role in the development of the project.

v. **Late submission.** Late submissions without a valid reason may be penalized (**10 marks per day**). If you experience serious issues (e.g., illness or accidents), email your lecturers as early as possible.

6 PLAGIARISM

Plagiarism is considered **cheating** and is strictly prohibited. This includes copying part of, or the entire, assignment from other students or from projects submitted in previous years. We will use plagiarism-checking tools to review submissions.

If you are found guilty of plagiarism (either copying from others **or** allowing others to copy your work), you will receive **zero marks** for the assignment.

Reference:
https://www.pygame.org/docs/tut/ImportInit.html